239. Sliding Window Maximum

Hard · Array · Queue · Sliding Window · Monotonic Queue

Approach — Monotonic Deque (Decreasing)

The solution uses a **monotonic decreasing deque** that stores indices of elements. At any point, the deque is maintained such that the indices it holds correspond to elements in decreasing order from front to back. This guarantees that the front of the deque always holds the index of the maximum element in the current window.

The deque `dq` is implemented manually using an array with two pointers `l` (front) and `r` (back). Before inserting a new index, all indices from the back whose corresponding values are smaller than or equal to the current element are removed — they can never be the maximum while the current element is in the window.

Key Insight: An element is useless if there is a newer, larger element still in the window. By evicting such elements from the back, the deque stays lean and the front always gives the window maximum in O(1). Each element is pushed and popped at most once, keeping the overall complexity linear.

Step-by-step Breakdown

Step 1 — Evict expired indices from the front

Before processing index `i`, check if the front of the deque `dq[l]` has fallen outside the current window (i.e. `dq[l] <= i-k`). If so, advance `l++` to remove the stale index.

Step 2 — Maintain the decreasing order from the back

Pop indices from the back of the deque while `nums[dq[r]] <= nums[i]`. These elements are smaller than `nums[i]` and are positioned to the left, so they will never be the window maximum as long as `i` is in the window. Evict them by decrementing `r`.

Step 3 — Push the current index

After cleaning the back, push the current index `i` onto the deque with `dq[++r] = i`. The deque now reflects a decreasing sequence of values from front to back, all within the valid window range.

Step 4 — Record the window maximum

Once `i >= k-1`, a full window of size k exists. The front of the deque `dq[l]` holds the index of the maximum element. Store `nums[dq[l]]` into `ans[i-k+1]` and continue.

Solution Code

```cpp
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        int n = nums.size();
        vector<int> dq(n), ans(n-k+1);
        int l = 0, r = -1;
        for(int i = 0; i < n; i++)
        {
            if(l <= r && dq[l] <= i-k) l++;
            while(l <= r && nums[dq[r]] <= nums[i]) r--;
            dq[++r] = i;
            if(i >= k-1) ans[i-k+1] = nums[dq[l]];
        }
        return ans;
    }
};
```

Complexity Analysis

TIME COMPLEXITY	SPACE COMPLEXITY
O(n)	**O(k)**
Each element is pushed onto the deque exactly once and popped at most once. All operations across the entire loop are O(n) total.	The deque holds at most k indices at any time — one per window slot. The output array is O(n-k+1) but is required by the problem.